ЛАБОРАТОРНА РОБОТА №4

СИСТЕМНЕ ПРОГРАМУВАННЯ

АСИНХРОННА МОДЕЛЬ ПРОГРАМУВАННЯ

Заяць Діани Юріївни

Хід виконання роботи:

4.1.1. Розробити програму Windows Forms Application, у якій запускається асинхронна задача кнопкою Start, причому вона виконується в окремому потоці, а елементи керування форми продовжують реагувати на дії користувача

Для виконання цього завдання було створено застосунок типу Windows Forms Application. У програмі реалізовано головну форму з кнопкою Start, яка запускає довготривалу операцію. Запуск реалізовано в обробнику натискання кнопки buttonStart_Click, де викликається асинхронний метод ProcessAsync. Важливо, що сама довготривала операція виконується не в головному потоці інтерфейсу, а у фоновому потоці за допомогою Task.Run. Завдяки цьому головний потік не блокується, а форма продовжує реагувати на взаємодію користувача.

У формі також додано групу перемикачів radioButton1-3, які використовуються як демонстрація того, що під час виконання довготривалої операції інтерфейс залишається активним. Тобто користувач може взаємодіяти з іншими елементами форми навіть тоді, коли асинхронна операція вже виконується. Це підтверджує, що задача дійсно винесена з UI-потоку.

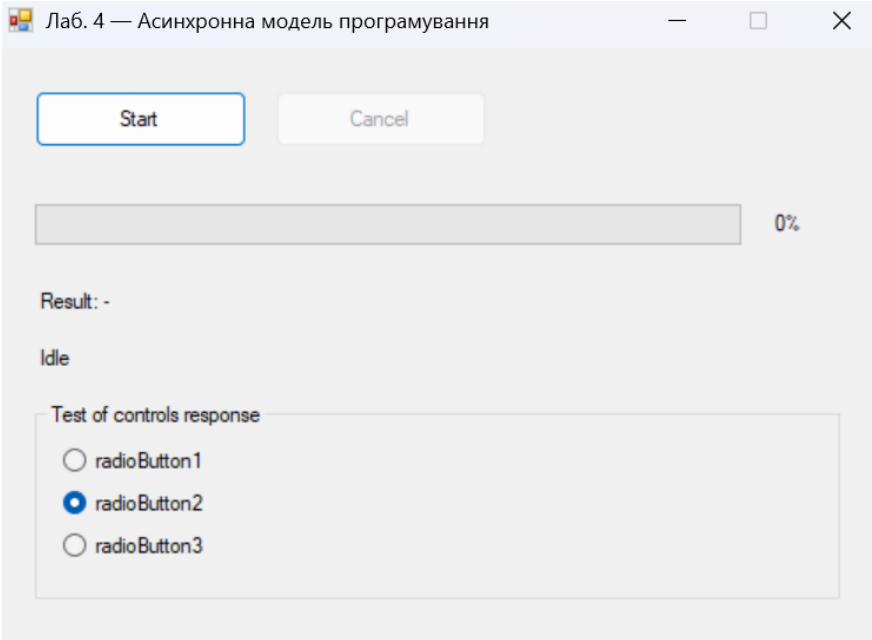

Рис. 4.1.1 – Головне вікно програми

```
// запускає асинхронну операцію та оновлює UI
private async void buttonStart_Click(object sender, EventArgs e)
{
    _cts?.Dispose(); // якщо попередній токен існує — звільняє його ресурси
    _cts = new CancellationTokenSource();

    var progress = new Progress<int>(percent =>
    {
        progressBar1.Value = percent;
        labelProgress.Text = percent + "%";
    });

    buttonStart.Enabled = false;
    buttonCancel.Enabled = true;
    labelStatus.Text = "Running...";

    try // основна асинхронна робота
    {
        const int count = 100;
        int result = await ProcessAsync(count, progress, _cts.Token).ConfigureAwait(true); /
        labelResult.Text = "Result: " + result;
        labelStatus.Text = "Completed";
    }
    catch (OperationCanceledException) // операцію було скасовано через токен
    {
        labelStatus.Text = "Cancelled";
        labelResult.Text = "Result: -";
        progressBar1.Value = 0;
        labelProgress.Text = "0%";
    }
    finally
    {
        buttonStart.Enabled = true;
        buttonCancel.Enabled = false;
        _cts?.Dispose();
        _cts = null;
    }
}
```

Рис. 4.1.2 – Фрагмент коду запуску асинхронної операції

```
// фонова довга операція: показує прогрес і повертає результат
private Task<int> ProcessAsync(int count, IProgress<int> progress, CancellationToken token)
{
    return Task.Run(async () => // не блокується юі, виконуємо в фоновому потоці
    {
        for (int i = 1; i <= 100; i++)
        {
            token.ThrowIfCancellationRequested(); // перевірка на скасування, якщо та - зупинка
            progress?.Report(i);
            await Task.Delay(50, token).ConfigureAwait(false); // якщо натиснули скасувати
        }

        int sum = 0;
        for (int j = 1; j <= count; j++)
        {
            token.ThrowIfCancellationRequested();
            sum += j;
        }

        return sum;
    }, token);
}
```

Рис. 4.1.3 – Фрагмент коду виконання задачі у фоновому потоці

4.2. Удосконалити програму так, щоб асинхронна операція повертала результат обчислень та відображала його на елементі керування форми

Для виконання другого завдання асинхронний метод було реалізовано з типом повернення Task<int>. Це означає, що після завершення довготривалої операції метод повертає ціле число як результат обчислень. У даному проєкті після завершення циклу обчислюється сума чисел від 1 до count, а отримане значення повертається як результат асинхронної задачі. Такий підхід повністю відповідає вимозі методичних вказівок щодо повернення значення з асинхронної операції.

Після завершення виконання методу ProcessAsync у обробнику buttonStart_Click результат зчитується через оператор await і виводиться на форму в елемент labelResult. Таким чином користувач бачить підсумок виконання довготривалої операції без необхідності чекати завершення програми або перевіряти значення вручну. Для тестового значення count = 100 результатом є сума 5050.

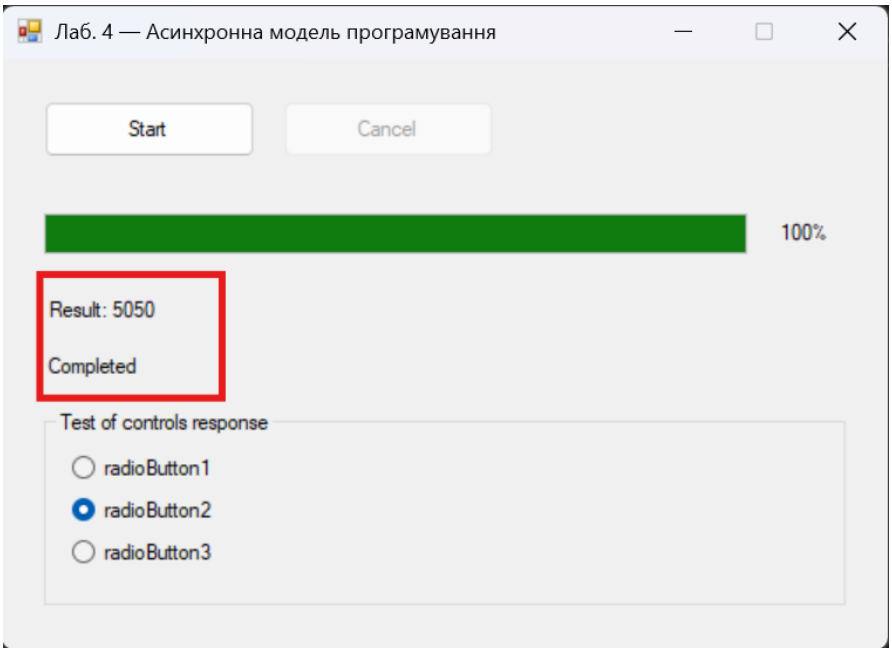

Рис. 4.2.1 – Відображення результату обчислень на формі

```
private Task<int> ProcessAsync(int count, IProgress<int> progress, CancellationToken token)
{
    return Task.Run(async () => // не блокується юі, виконуємо в фоновому потоці
    {
        for (int i = 1; i <= 100; i++)
        {
            token.ThrowIfCancellationRequested(); // перевірка на скасування, якщо та - зупин
            progress?.Report(i);
            await Task.Delay(50, token).ConfigureAwait(false); // якщо натиснули скасувати
        }

        int sum = 0;
        for (int j = 1; j <= count; j++)
        {
            token.ThrowIfCancellationRequested();
            sum += j;
        }

        return sum;
    }, token);
```

Рис. 4.2.2 – Фрагмент коду з типом Task<int>

```
try // основна асинхронна робота
{
    const int count = 100;
    int result = await ProcessAsync(count, progress
    labelResult.Text = "Result: " + result;
    labelStatus.Text = "Completed";
}
```

Рис. 4.2.3 – Фрагмент коду виведення результату на форму

4.3. Доповнити функціонал програми можливістю відображати хід виконання операції у текстовій та графічній формах

Для виконання цього завдання в програмі було реалізовано механізм передачі прогресу виконання за допомогою інтерфейсу IProgress<int> та класу Progress<int>. У момент запуску створюється об’єкт прогресу, який пов’язаний із елементами інтерфейсу. Під час виконання довготривалої операції в циклі викликається метод Report(i), що передає поточне значення прогресу у відсотках. Методичка прямо вказує використовувати IProgress<T> для інформування про хід асинхронної операції.

У формі прогрес відображається двома способами. По-перше, текстово — через елемент labelProgress, у якому показується поточний відсоток виконання. По-друге, графічно — через progressBar1, у якого встановлено максимальне значення 100. Таким чином користувач одночасно бачить числовий і візуальний індикатор виконання задачі. Це повністю виконує третю вимогу лабораторної роботи.

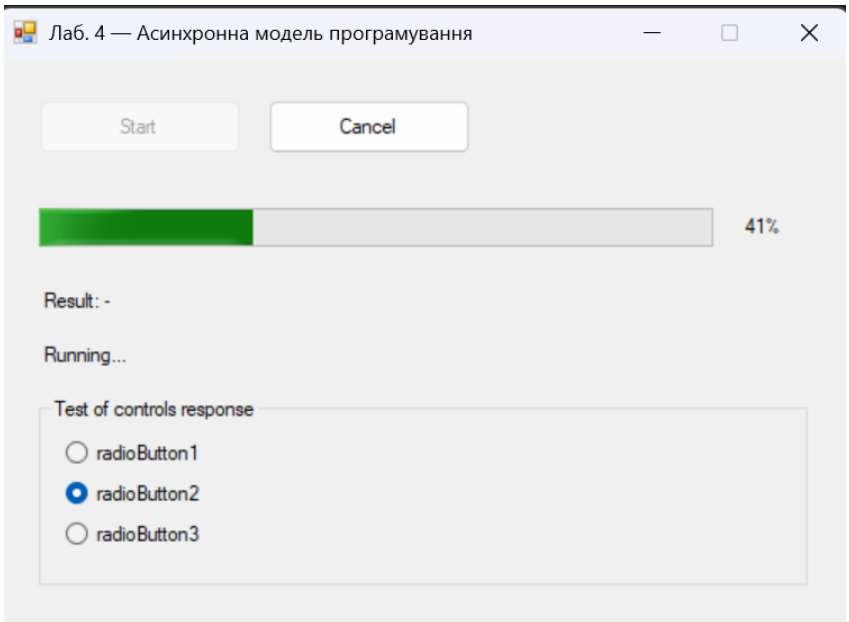

Рис. 4.3.1 – Відображення прогресу виконання у програмі

```
private async void buttonStart_Click(object sender, EventArgs e)
{
    _cts?.Dispose(); // якщо попередній токен існує — звільняє його ресурс
    _cts = new CancellationTokenSource();

    var progress = new Progress<int>(percent =>
    {
        progressBar1.Value = percent;
        labelProgress.Text = percent + "%";
    });

    buttonStart.Enabled = false;
    buttonCancel.Enabled = true;
    labelStatus.Text = "Running...";
```

Рис. 4.3.2 – Фрагмент коду створення IProgress<int>

```
// фонова довга операція: показує прогрес і повертає результат
private Task<int> ProcessAsync(int count, IProgress<int> progress,
{
    return Task.Run(async () => // не блокується ui, виконуємо в ф
    {
        for (int i = 1; i <= 100; i++)
        {
            token.ThrowIfCancellationRequested(); // перевірка на
            progress?.Report(i);
            await Task.Delay(50, token).ConfigureAwait(false); //

        }

        int sum = 0;
        for (int j = 1; j <= count; j++)
        {
            token.ThrowIfCancellationRequested();
            sum += j;
        }

        return sum;
    }, token);
}
```

Рис. 4.3.3 – Фрагмент коду передачі прогресу

4.4. Розширити функціонал програми можливістю дочасно перервати виконання довготривалої операції кнопкою Cancel

Для реалізації дострокового скасування було використано механізм CancellationTokenSource і CancellationToken. Після натискання кнопки Start створюється новий об’єкт CancellationTokenSource, який використовується лише для поточного запуску задачі. Його токен передається в асинхронний метод ProcessAsync, де під час кожної ітерації циклу перевіряється, чи не було запрошено скасування виконання. Такий підхід відповідає методичним вказівкам щодо реалізації відміни асинхронних операцій.

Кнопка Cancel викликає метод Cancel() у джерела токена, після чого асинхронна операція завершується достроково. У програмі ця ситуація обробляється окремо через OperationCanceledException, тому скасування не

вважається помилкою. Після цього на формі оновлюється статус на Cancelled, результат скидається, а кнопки повертаються у початковий стан. Це забезпечує коректне завершення програми без зависання та аварійного завершення.

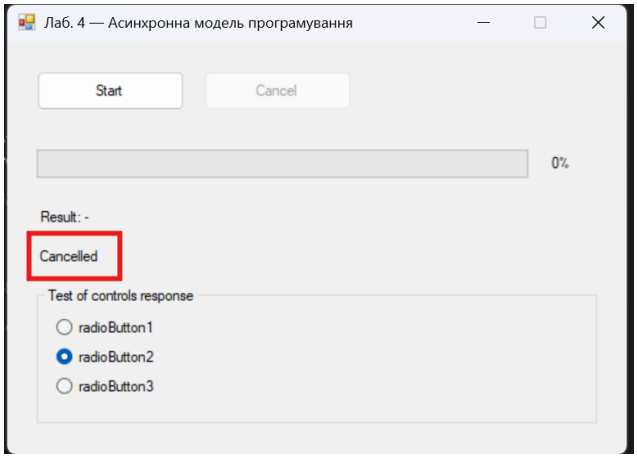

Рис. 4.4.1 – Робота кнопки Cancel

```
// запускає асинхронну операцію та оновлює UI
private async void buttonStart_Click(object sender, EventArgs e)
{
    _cts?.Dispose(); // якщо попередній токен існує – звільняє його ресурси
    _cts = new CancellationTokenSource();

    var progress = new Progress<int>(percent =>
    {
        progressBar1.Value = percent;
        labelProgress.Text = percent + "%";
    });

    buttonStart.Enabled = false;
    buttonCancel.Enabled = true;
    labelStatus.Text = "Running...";

    try // основна асинхронна робота
    {
        const int count = 100;
        int result = await ProcessAsync(count, progress, _cts.Token).ConfigureAwait(true);
        labelResult.Text = "Result: " + result;
        labelStatus.Text = "Completed";
    }
    catch (OperationCanceledException) // операцію було скасовано через токен
    {
        labelStatus.Text = "Cancelled";
        labelResult.Text = "Result: -";
        progressBar1.Value = 0;
        labelProgress.Text = "0%";
    }
    finally
    {
        buttonStart.Enabled = true;
        buttonCancel.Enabled = false;
        _cts?.Dispose();
        _cts = null;
    }
}
```

Рис. 4.4.2 – Фрагмент коду створення CancellationTokenSource та обробки скасування

```
    return Task.Run(async () => // не блокується ui, виконуємо в
    {
        for (int i = 1; i <= 100; i++)
        {
            token.ThrowIfCancellationRequested(); // перевірка н
            progress?.Report(i);
            await Task.Delay(50, token).ConfigureAwait(false); /
        }

        int sum = 0;
        for (int j = 1; j <= count; j++)
        {
            token.ThrowIfCancellationRequested();
            sum += j;
        }

        return sum;
    }, token);
}
```

Рис. 4.4.3 – Фрагмент коду перевірки скасування

Висновок

У ході виконання лабораторної роботи було розроблено Windows Forms застосунок мовою C#, який демонструє використання асинхронної моделі програмування. У програмі реалізовано запуск довготривалої операції у фоновому потоці за допомогою Task.Run, очікування її завершення через async/await, повернення результату обчислень типу Task<int>, відображення прогресу у текстовій та графічній формах за допомогою IProgress<int>, а також можливість дострокового скасування виконання через CancellationTokenSource і CancellationToken. У результаті було підтверджено, що асинхронний підхід дозволяє виконувати довготривалі операції без блокування інтерфейсу користувача, забезпечуючи зручність і коректність роботи програми.